

Rapport de TP

Application CRUD de Gestion des Utilisateurs

Full-Stack React.js + Express.js

Module : Web Sémantique (SDIA S2)

Réalisé par : Ouaoouche Hicham

Encadré par : Prof Oumayma AGHERAI

1. Présentation du projet

Ce travail pratique consiste à développer une application web complète de gestion des utilisateurs, reposant sur une architecture Full-Stack JavaScript moderne. Le projet est structuré en deux parties distinctes mais complémentaires :

- Un back-end développé avec Node.js et le framework Express.js, exposant une API REST sur le port 5000
- Un front-end développé avec React.js, accessible sur le port 3000

L'application implémente les quatre opérations fondamentales du CRUD (Create, Read, Update, Delete) sur une liste d'utilisateurs. Chaque utilisateur est caractérisé par un identifiant unique, un nom, une adresse e-mail et un rôle (Admin, Utilisateur ou Modérateur).

Les données sont stockées en mémoire dans un tableau JavaScript, ce qui simplifie le projet et le rend idéal pour un exercice pédagogique, tout en conservant une architecture proche d'une vraie application web.

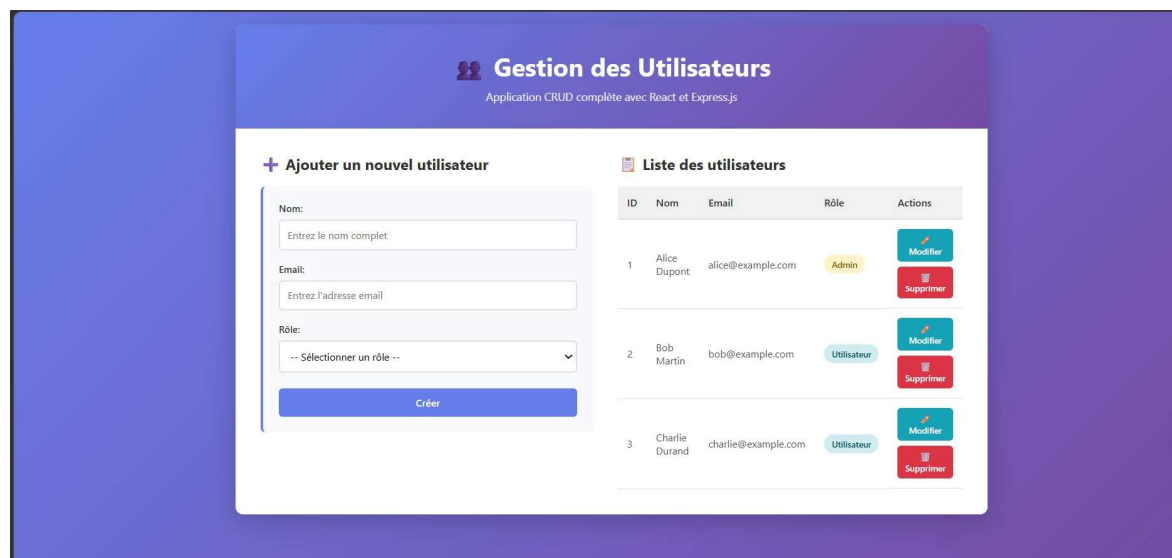


Figure 1 — Interface principale de l'application CRUD

2. Objectifs du TP

L'objectif principal de ce TP est de réaliser une application complète en connectant un front-end React à une API REST Express. Le projet met en pratique les compétences suivantes :

- Création et configuration d'une API REST avec Express.js
- Gestion des requêtes HTTP (GET, POST, PUT, DELETE)
- Utilisation du hook useEffect() pour le chargement des données au démarrage

- Gestion d'un formulaire unique servant à la fois à l'ajout et à la modification
- Affichage d'une liste interactive avec des actions dynamiques
- Communication entre front-end et back-end via la Fetch API
- Mise en place du middleware CORS pour les requêtes cross-origin

3. Architecture et structure du projet

Le projet suit une architecture client-serveur claire, avec une séparation nette entre le front-end et le back-end. Voici la structure des fichiers :

```
Projet_tp/
├── backend/
│   ├── package.json      # Dépendances : express, cors
│   └── server.js         # Serveur Express + routes CRUD
├── frontend/
│   ├── package.json      # Dépendances : react, react-dom
│   ├── public/
│   │   └── index.html
│   └── src/
│       ├── App.js        # Composant principal React
│       ├── App.css       # Styles personnalisés
│       └── index.js       # Point d'entrée React
└── README.md
```

4. Partie Back-End — API REST Express.js

4.1 Configuration du serveur

Le serveur Express est configuré dans le fichier `server.js` et écoute sur le port 5000. Deux middlewares essentiels sont mis en place :

- `express.json()` : permet de lire les corps de requêtes au format JSON
- `cors()` : autorise le front-end React (`localhost:3000`) à communiquer avec le serveur

```
const express = require('express');
const cors = require('cors');
const app = express();
const PORT = 5000;

app.use(express.json());
app.use(cors({ origin: 'http://localhost:3000', methods:
['GET', 'POST', 'PUT', 'DELETE'] }));
```

4.2 Base de données en mémoire

Les utilisateurs sont stockés dans un tableau JavaScript. Trois utilisateurs sont pré-chargés au démarrage :

```
let utilisateurs = [
  { id: 1, nom: 'Alice Dupont', email: 'alice@example.com', role: 'Admin' },
  { id: 2, nom: 'Bob Martin', email: 'bob@example.com', role: 'Utilisateur' },
  { id: 3, nom: 'Charlie Durand', email: 'charlie@example.com', role: 'Utilisateur' },
];
let prochainId = 4;
```

4.3 Routes de l'API

L'API expose quatre routes REST couvrant toutes les opérations CRUD :

Méthode	Route	Description	Code HTTP
GET	/users	Récupérer tous les utilisateurs	200 OK
POST	/users	Créer un nouvel utilisateur	201 Created
PUT	/users/:id	Modifier un utilisateur existant	200 OK
DELETE	/users/:id	Supprimer un utilisateur	200 OK

```
{
  "users": [
    {
      "id": 1,
      "nom": "Alice Dupont",
      "email": "alice@example.com",
      "role": "Admin"
    },
    {
      "id": 2,
      "nom": "Bob Martin",
      "email": "bob@example.com",
      "role": "Utilisateur"
    },
    {
      "id": 3,
      "nom": "Charlie Durand",
      "email": "charlie@example.com",
      "role": "Utilisateur"
    }
  ]
}
```

Figure 2 — Réponse JSON de l'API lors de l'appel GET /users

4.4 Gestion des erreurs

Chaque route implémente une validation basique des données reçues. En cas de champs manquants, le serveur renvoie un code HTTP 400 avec un message d'erreur explicite. En cas d'utilisateur non trouvé lors d'une mise à jour ou d'une suppression, un code 404 est retourné.

5. Partie Front-End — React.js

5.1 Architecture du composant

Le front-end est développé dans un seul composant principal App.js. Il utilise le hook useState pour gérer l'état local et useEffect pour déclencher le chargement initial des utilisateurs :

- utilisateurs : tableau contenant la liste des utilisateurs chargés depuis l'API
- formulaire : objet contenant les valeurs des champs (nom, email, rôle, id)

- enModification : booléen indiquant si le formulaire est en mode édition
- message : objet contenant le texte et le type du message de retour (succès ou erreur)

5.2 Interface principale

L'interface est divisée en deux sections côte à côte :

- À gauche : un formulaire permettant d'ajouter ou de modifier un utilisateur
- À droite : un tableau affichant la liste des utilisateurs avec leurs informations et leurs actions

5.3 Formulaire d'ajout

Le formulaire d'ajout contient trois champs : le nom complet, l'adresse e-mail et le rôle sélectionné via un menu déroulant. Un bouton "Créer" déclenche l'envoi d'une requête POST vers l'API.

+ Ajouter un nouvel utilisateur

Nom:

Email:

Rôle:



Créer

Figure 3 — Formulaire d'ajout d'un nouvel utilisateur

5.4 Liste des utilisateurs

La liste des utilisateurs est affichée sous forme de tableau avec les colonnes ID, Nom, Email, Rôle et Actions. Le rôle est mis en évidence par un badge coloré. Chaque ligne dispose de deux boutons d'action.

 **Liste des utilisateurs**

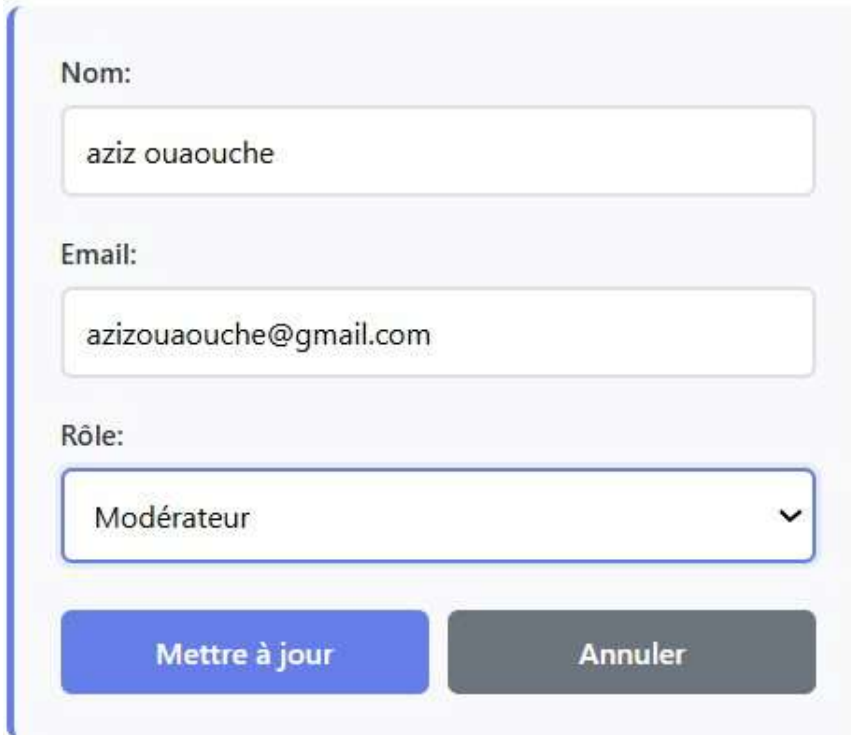
ID	Nom	Email	Rôle	Actions
1	Alice Dupont	alice@example.com	Admin	Modifier Supprimer
2	Bob Martin	bob@example.com	Utilisateur	Modifier Supprimer
3	Charlie Durand	charlie@example.com	Utilisateur	Modifier Supprimer
4	ouaouche hicham	hichamouaouche@gmail.com	Admin	Modifier Supprimer

Figure 4 — Liste des utilisateurs après ajout d'un quatrième utilisateur

5.5 Modification d'un utilisateur

Lorsque l'utilisateur clique sur le bouton "Modifier", le formulaire bascule en mode édition. Les champs sont automatiquement pré-remplis avec les données de l'utilisateur sélectionné. Le bouton "Créer" devient "Mettre à jour" et un bouton "Annuler" apparaît pour quitter le mode édition.

Modifier un utilisateur



Nom:

Email:

Rôle:

▼

Figure 5 — Formulaire en mode modification

5.6 Suppression d'un utilisateur

La suppression d'un utilisateur est protégée par une boîte de dialogue de confirmation native du navigateur (window.confirm). L'utilisateur doit confirmer son intention avant que la requête DELETE soit envoyée au serveur.

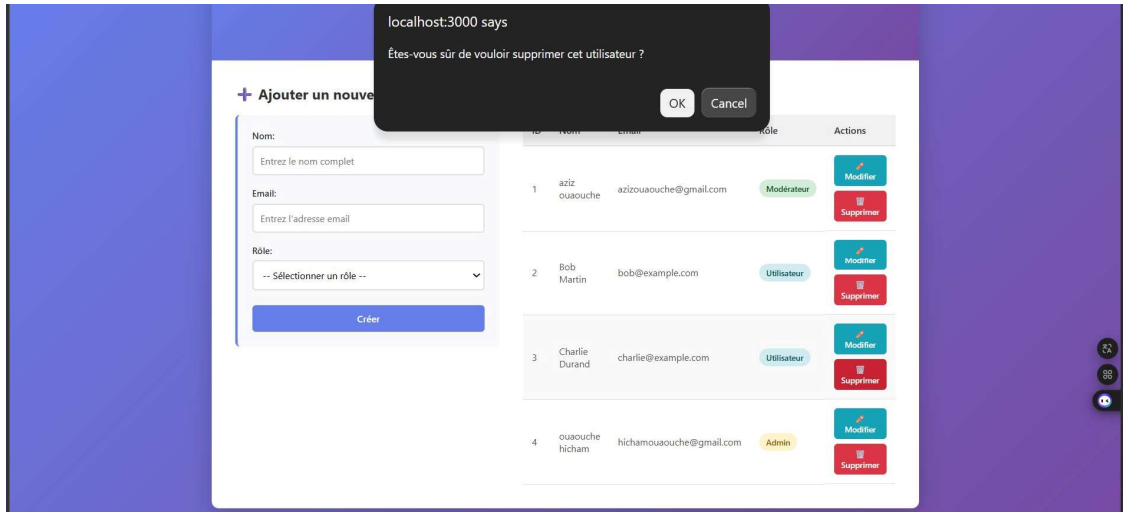


Figure 6 — Dialogue de confirmation de suppression

6. Technologies utilisées

Technologie	Catégorie	Rôle dans le projet
Node.js	Environnement Back-End	Exécution JavaScript côté serveur
Express.js	Framework Back-End	Création de l'API REST
CORS	Middleware	Autorisation des requêtes cross-origin
React.js 18	Framework Front-End	Construction de l'interface utilisateur
Fetch API	Client HTTP	Communication entre front et back
CSS personnalisé	Style	Design de l'interface
npm	Gestionnaire de paquets	Gestion des dépendances

7. Installation et lancement

7.1 Prérequis

- Node.js installé sur la machine
- npm disponible dans le terminal
- Deux terminaux ouverts simultanément

7.2 Lancement du back-end

```
cd backend
npm install
npm start

# Résultat attendu :
# 🚀 Serveur Express démarré sur http://localhost:5000
# 📡 En attente de requêtes API...
```

7.3 Lancement du front-end

```
cd frontend
npm install
npm start

# L'application s'ouvre automatiquement sur :
# http://localhost:3000
```

Les deux serveurs doivent être lancés simultanément : le back-end sur le port 5000 et le front-end sur le port 3000.

8. Tests et résultats

Les tests réalisés démontrent le bon fonctionnement de l'ensemble des fonctionnalités :

- L'affichage de la liste fonctionne correctement au démarrage et récupère les données via GET /users
- L'ajout d'un utilisateur envoie une requête POST et met à jour le tableau immédiatement
- La modification recharge les données mises à jour sans rechargement complet de la page
- La suppression demande une confirmation et supprime l'entrée via DELETE /users/:id
- Les messages de succès et d'erreur s'affichent temporairement pendant 4 secondes
- La validation des champs côté client empêche l'envoi de formulaires incomplets

9. Conclusion et perspectives

Ce TP a permis de mettre en place une application CRUD complète, structurée en deux parties distinctes et facile à maintenir. L'architecture choisie — back-end Express séparé du front-end React — reflète les pratiques courantes du développement web moderne.

L'utilisation d'un tableau en mémoire rend le projet simple à comprendre dans un contexte pédagogique, tout en conservant une architecture réaliste. Le projet constitue une bonne base pour aborder des développements plus avancés.

Améliorations possibles

- Remplacer le stockage en mémoire par une base de données persistante (MongoDB, MySQL, PostgreSQL)
- Ajouter un système d'authentification avec JWT (JSON Web Tokens)
- Implémenter la pagination de la liste pour gérer un grand nombre d'utilisateurs
- Ajouter une fonctionnalité de recherche et de filtrage par rôle
- Déployer l'application sur un service cloud (Vercel, Railway, Render)
- Écrire des tests unitaires et d'intégration avec Jest et Testing Library